A Service-Oriented Digital Twin Architecture for Seamless Reliability Verification of IoT Systems

Janik-Vasily Benzin[1](✉), Jürgen Mangler[1], and Stefanie Rinderle-Ma[1]

Technical University of Munich, TUM School of Computation, Information and Technology, Garching, Germany
{janik.benzin,juergen.mangler,stefanie.rinderle-ma}@tum.de

Abstract. Internet of Things (IoT) systems are distributed collections of embedded systems that are connected over the Internet or similar networks. A key property of IoT systems is the need for high reliability throughout their lifespan under physical constraints, e.g., material properties, and natural degradation of system components. Simulation and digital twin approaches can quantify statistical assurances at design time and runtime respectively to verify reliability of IoT systems. However, existing approaches lack integration through a multi-paradigm modelling approach for seamless design time and runtime reliability verification, sole use of mature standards and protocols for interoperability and easy extension, and state-of-the-art statistical model checking (SMC) techniques for sound and efficient reliability verification. Consequently, we propose a service-oriented digital twin architecture and implement the architecture in a production-grade, interoperable, and radically distributed scale out process engine that integrates both design and runtime verification in a single model. We demonstrate our architecture and implementation for four IoT systems and report statistical reliability assurances through sound and efficient SMC.

Keywords: Digital twin · Service-Oriented Architecture · Reliability · Execution · Internet of Things · Statistical Model Checking

1 Introduction

Internet of Things (IoT) systems are deployed on heterogeneous IT infrastructures of their application, e.g., healthcare for patient monitoring [12]. The diversity of *embedded systems* [19] that comprise the IoT system can be high and the set of embedded systems in an IoT system can change over time. An embedded system has a discrete control part (microcontroller) and a physical dynamics part (physical device). While embedded systems are characterized by constrained runtime environments and face natural degradation and device failures, they must be reliable over their lifespan to continuously perform their function. For example, a collection of thermostats is reliable, if they maintain the room temperature within a prescribed range over a year with a high probability.

Existing approaches to verify reliability through *statistical model checking* (SMC) [5] can be classified along the lifecycle of the IoT system. Design time

approaches verify reliability before the IoT system is deployed to production by simulating the system behavior through a system model. The most powerful models are *stochastic hybrid systems* (SHS) [19] that combine stochastic physical dynamics of the physical device with stochastic, discrete control behaviour of the microcontroller. As such, SHS combine *ordinary differential equations* (ODEs) for physical dynamics with state-based models for discrete behaviour. By contrast, runtime approaches verify reliability while the IoT system is running. To that end, runtime approaches use a *digital twin* (DT) [20] of the IoT system, i.e., its dynamic digital representation with the ability to control the IoT system [20]. Because a DT is connected to the real IoT system, runtime approaches do not model the IoT system behaviour, but instead focus on the service design, development, and orchestration to implement and deploy the DT [27,13,9].

Currently, design and runtime approaches are separated since the conceptual models (SHS vs. service orchestration) and technical implementations (monolithic vs. service-oriented) differ significantly. These conceptual and technical differences aggravate model reuse from design time approaches for developing the DT at runtime and make it impossible to use the DT for design time reliability verification. However, a seamless integration of formal design time model and DT bears the great benefit of reduced implementation effort under formal reliability guarantees. Hence, to overcome the challenge of seamless reliability verification and to unlock its potential, we propose **Twin-CPEE**, which is a multi-paradigm DT that combines the *cloud process execution engine* (CPEE) [22] for execution and service orchestration at runtime with SHS for simulation at design time. Our proposal consists of a DT service-oriented architecture and service pattern to implement multi-paradigm DTs [4] that enable seamless reliability verification through SMC. The multi-paradigm DT applies Business Process Modeling and Notation (BPMN)[1] as de facto standard for service orchestration and SHS as the theoretical foundation for simulating both stochastic physical dynamics and discrete control. Twin-CPEE builds on the CPEE due to its use of mature standards [22], its lightweight and flexible deployment to embedded systems [30], and success in industry[2]. Finally, a DT that is developed with Twin-CPEE enables the recording of *IoT-enriched logs* [23] that contain service execution and physical variable information at design time and runtime.

In Section 2, we state requirements for a multi-paradigm DT and related work. In Sect. 3, we present the architecture and service pattern of Twin-CPEE. In Sect. 4, we demonstrate Twin-CPEE by verifying reliability of three IoT systems through SMC. In Sect. 5, we conclude and give an outlook.

2 Related Work and Requirements

Design time approaches and their formal modelling language to simulate the IoT system stem from different domains, i.e., from formal methods and business pro-

[1] `bpmn.org`

[2] 1.689.504 downloads, see `https://rubygems.org/profiles/eTM`, accessed 2025-07-21; 16 process orchestrations run operatively at 7 manufacturing companies.

cess management (BPM). Formal methods approaches (denoted by ^{f} in Tab. 1) advance the theoretical frontier on model expressiveness and statistical model checking. BPM approaches are denoted by ^{b}. Existing work on extending BPMN to represent IoT systems [12] at design time often lacks formal semantics for reliability verification or focuses on certain aspects such as *complex event processing* [21] without providing a running simulation model. For runtime approaches, we focus on digital twins of IoT systems that are based on some process modeling language to orchestrate the embedded systems. Thus, literature on digital twins of single embedded systems from the engineering domain is out of scope [20]. Additionally, we focus on approaches that produce execution logs in a standard format, e.g., eXtensible Event Stream (XES) or a format of similar quality [16] for subsequent analysis. The resulting set of existing approaches, the requirements set out by existing approaches at design and runtime of the DT, and an assessment of existing approaches along the requirements are depicted in Tab. 1.

Table 1: Requirements and Existing Approaches.

General				Representation				Architecture	
Lifecycle	Name	Ref.	Year	Cont.	Data	Hybrid	Stoch. (S/D)	SOA	Stand.
Design	IoT-BP-DT^b	[28]	2023	✗	✓	✗	✗/✗	✓	✗
	HYPEG^f	[26]	2017	✓	✗	✓	✓/✗	✗	✗
	COSMOS^f	[3]	2015	✓	✗	✗	✓/✗	✗	✗
	UPPAAL SMC^f	[11]	2015	✓	✗	✓	✓/✗	✗	✗
	PRISM^f	[18]	2011	✓	✓	✗	✓/✓	✗	✗
	CPNTools^b	[17]	2007	✓	✓	✗	✓/✓	✗	✗
Runtime	Smarter	[27]	2025	✓	✓	✗	✗/✗	✓	✓
	FloBP	[13]	2024	✓	✓	✗	✗/✗	✓	✓
	Prowin	[9]	2023	✓	✓	✗	✗/✗	✗	✓
	FAME	[8]	2023	✓	✓	✗	✗/✗	✗	✓
	CPEE	[22]	2022	✓	✓	✗	✗/✗	✓	✓
	IoT-enhanced BP	[29]	2022	✓	✓	✗	✗/✗	✓	✗
	DCH-JBPM	[6]	2019	✓	✗	✗	✗/✗	✗	✗
	makeSense	[25]	2019	✓	✓	✗	✗/✗	✗	✗
	Smart Device	[31]	2018	✓	✓	✗	✗/✗	✓	✓
	ROA	[10]	2015	✓	✗	✗	✗/✗	✓	✗
	ESPT	[1]	2014	✓	✗	✗	✗/✗	✗	✗

In Tab. 1, representation requirements comprise a *cont*inuous time notion in $\mathbb{R}_{\geqslant 0}$ and *data*flow of physical, continuous and arbitrarily typed variables represented explicitly, e.g., through a shared variable concept with explicit data manipulation, or not. Moreover, similar to the theory of *hybrid* systems, an approach can either allow simulation of physical processes, e.g., a heating of a kettle, through their corresponding ordinary differential equation systems or not. Likewise, *stoch*astic elements are increasingly added to hybrid systems resulting in stochastic hybrid systems [19], e.g., a sensor variable is assigned a sampled value from a *s*tatic probability distribution to model noisy sensors or from a *d*ynamic probability distribution that changes its shape depending on other variables such as time to model degrading, noisy sensors. Regarding architectural

requirements, approaches can benefit from the advantages of service-oriented architectures (*SOA*) or not. Lastly, an approach can either have open interfaces with *stand*ardized protocols or not.

Overall, none of the existing approaches covers both, design and runtime representation for simulation and execution of DTs. This results in the inability to either reliability verification or DT execution. Hence, in the following, we present the multi-paradigm Twin-CPEE approach integrating both, design and runtime representation and implementation of a DT in a seamless manner.

3 Twin-CPEE

We present the service-oriented architecture (cf. Sect. 3.1) and the associated service pattern for implementing multi-paradigm DTs of IoT systems (cf. Sect. 3.2).

3.1 A Service-Oriented Digital Twin Architecture

Due to the heterogeneous IT infrastructures in which IoT systems are deployed, the diverse range of embedded systems, and the dynamic nature of the IoT system, a service-oriented architecture with an orchestrating process engine is well-suited as a DT of an IoT system. The decoupled, modular, and distributed layout of services in the DT architecture of Twin-CPEE is depicted in Fig. 1. On a high-level, the architecture consists of five main groups of (micro-)services (denoted by I-V in Fig. 1). Group I (emphasized in orange in Fig. 1) is the DT service pattern that is presented in detail in the next section. Group II shows the REST operation interface of process instances (as denoted in violet in I) with standard wrapper microservices for connecting communication protocols of embedded systems in more detail than in group I. Group III is the CPEE [22] with its microservices abstracted into a single service. The CPEE supports the representation of the service orchestrations in BPMN that are internally serialized into Refined Process Structure Trees with well-defined execution semantics. Group IV shows the client with two different user interfaces (UIs). Lastly, group V shows supporting microservices that enable offline and online analysis.

As a central, logical container of current sensor and actuator values, the `DT container` in group I is a simple microservice dedicated to only hold the current values. The `DT container` in conjunction with `physical`, `environment`, or `control` process instances is sufficient to model, simulate, and execute an IoT system and its DT. Indepent of the lifecycle, we distinguish different communication pathways that must always be adhered to. Each process instance freely communicates (1) with the `DT container` via REST (cf. group I in Fig. 1), (2) with wrapper microservices (cf. II in Fig. 1) via REST and (3) with the `CPEE` (cf. III in Fig. 1) via *Pub/Sub inter-process communication* (IPC). Via Pathway (1), the `DT container` receives new sensor and actuator values or sends the current values upon request from or to any of the `physical`, `environment`, or `control` process instances respectively. Via pathway (2), a process instance can be connected to embedded systems. To that end, a process instance's operation

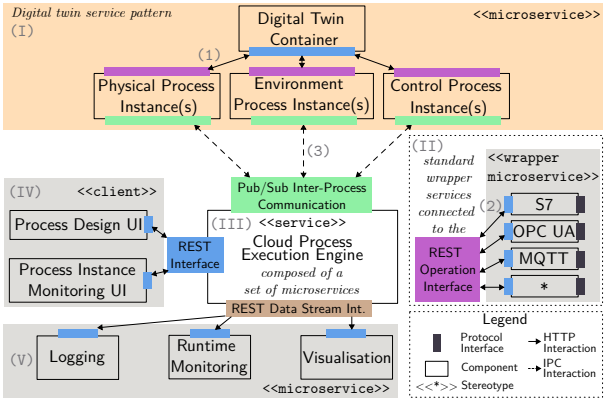

Fig. 1: The service-oriented digital twin architecture of Twin-CPEE.

interface is connected to Twin-CPEE's extensible set of wrapper microservices, which currently wrap the *S7*, *OPC UA* and *MQTT* protocols. Hence, the wrapper microservices allow easy integration with a large set of embedded systems for execution of the DT at runtime through tasks that call the respective wrapper service. The third pathway (3) of exists for highly performant IPC communication between process instances and the CPEE.

Importantly, the architecture prohibits direct communication between process instances for two reasons. First, a single, indirect communication pathway between process instances via the `DT container` is transparent and easily maintainable. Second, any values sent to the `DT container` represent the current state of the IoT system such that monitoring becomes straightforward compared to deciding upon each direct communication between process instances whether this communication describes the IoT system's current state or not.

While a CPEE process model is designed in the `process design UI` (cf. Fig. 1 IV), the UI communicates via REST with the CPEE to change the model. As soon as a CPEE model is instantiated via a REST call to the CPEE's REST interface, the CPEE transpiles the respective CPEE process model into executable Ruby, JavaScript, or platform-dependent bytecode through transpiling into Rust [30] and compiling for the host platform. The executable code is executed in its own OS process of the respective host's runtime environment, where a host could be a server, e.g., for the `environment` or `control` process instance, or an embedded system, e.g., for the `physical` process instance. Because the Rust compiler targets different host platforms[3], most embedded systems can at least execute their bytecode of a process instance. To coordinate its execution with the CPEE, a process instance instantaneously notifies the process engine about its state changes, e.g., from running to paused, and about new events, e.g., that a task is started or it received new values from the `DT container`. The CPEE sends state changes to the `process instance monitoring UI` (cf. Fig. 1

[3] `https://doc.rust-lang.org/nightly/rustc/platform-support.html`

IV) and the `runtime monitoring` (cf. Fig. 1 V) microservice for instance- and CPEE-level monitoring respectively. It sends events to the `logging` microservice for persistence in standard XES logs (cf. Fig. 1 V) and to the `visualisation` microservice to visually monitor evolution of sensor and actuator values.

Overall, the service-oriented DT architecture of Twin-CPEE consists of a set of microservices in which each microservice corresponds to a clearly delineated functionality required for simulating and executing multi-paradigm DTs of IoT systems. In addition, the architecture defines a set of mainly REST-based communication pathways.

3.2 Multi-paradigm DT Service Pattern for Reliability Verification

In this section, we present the multi-paradigm DT service pattern of Twin-CPEE and implement the service pattern in an example. The multi-paradigm DT service pattern is depicted in Fig. 2, refines how the three different types of process instances are allowed to communicate with the `DT container`, and specifies what process instance can read or write what type of variable via pathway (1) (cf. last section). The pattern realizes a DT of an IoT system with a single embedded system. Since the `physical` and `control` process pattern are independent of the corresponding embedded system, DTs of IoT systems with multiple embedded systems are implemented through applying the `physical` and `control` process pattern for each embedded system respectively. The process that writes a variable determines its namespace: `physical`, `environment`, and `control` processes write `sensor/<var_id>`, `environment/<var_id>`, and `actuator/<var_id>` variables respectively. In the following, we specify read/write operations at design time. We refer to an implementation of the pattern at design time as design time DT. Afterwards, we outline the changes required to move the DT from design to runtime yielding a runtime DT.

Physical read/write. The `physical` process can read all three types of variables (cf. "full" denoted in purple in Fig. 2), i.e., sensor variables $sensor1$-$sensorN$ and actuator variables $actuator1$-$actuatorN$ of a single embedded system as well as environment variables $env1$-$envN$ of the IoT system. Hence, all three types of variables with their current values taken together capture the current state of the IoT system. The reading of all three types of variables is executed by the task labelled with "Read" in Fig. 2 b. The reading task has CPEE's subscription functionality [22] such that the `DT container` calls the callback url of the subscribed reading task each time a new value is written to any of the variables to which the reading task is subscribed. Thus, the reading task is running as long as the DT is running.

Due to taking a multi-paradigm approach to combine both SHS and BPMN (cf. Sect. 1), the `physical` process allows to simulate physical dynamics with ODEs. Since embedded systems are *reactive systems* [15] that run indefinitely, an indefinite loop is executed in parallel to the reading task (cf. Fig. 2 b) with a certain frequency. Each loop cycle, the `physical` process waits for a defined time, e.g., 1s. Following a *mode-based* specification of SHS [15] that model embedded systems [19], every cycle a decision is taken that depends on the control mode,

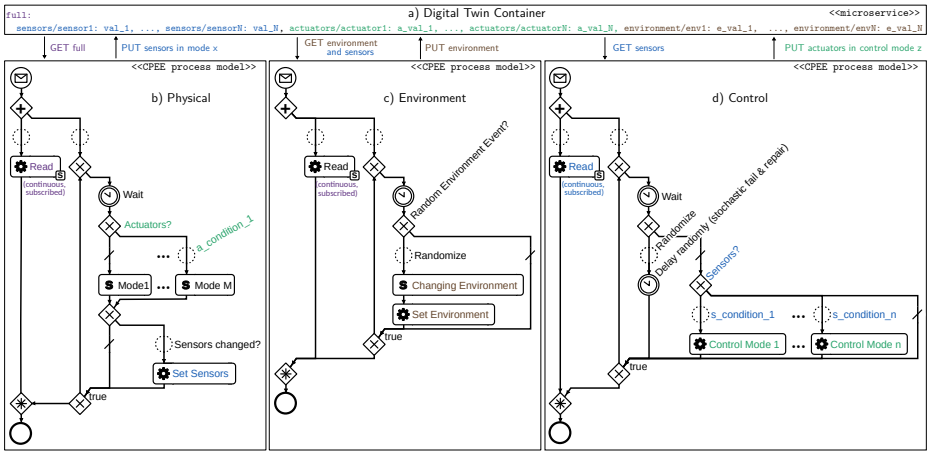

Fig. 2: The multi-paradigm digital twin service pattern of Twin-CPEE.

e.g., a smart light is turned on or an automated motor is set to a discrete acceleration. The control mode is logically expressed as a condition over the current actuator variable values (cf. *a_condition_*1 in Fig. 2 b). At design time, each branch of the decision has a script task that is labelled with the corresponding mode, e.g., "Mode 1" in Fig. 2. The mode's respective script task computes a solution to an ODE that specifies the dynamic behaviour of the real-world physical process as captured by the sensors. Similar to SHS, the `physical` process simulates the real-world physical dynamics by computing the ODE solution.

Control read/write. The `control` process can read sensor values through a similar reading task as the `physical` process and also has the indefinite cycle of reactive systems (cf. Fig. 2 d) for a remote control in IoT systems. Depending on the sensor values (cf. *s_condition_*1), in each cycle a decision is taken that determines the next control mode, which corresponds to new actuator values written to the `DT container` through the corresponding task, e.g., "Control Mode 1". For reliability verification, the `control` process has a stochastic fail and repair, which randomly occurs in every cycle and can be dynamically distributed according to any kind of probability distribution. Upon failure, a random delay models the time of failure and repair. Thus, the `control` process includes randomness similar to SHS. At runtime, the stochastic fail and repair must be removed from the `control` process, because degradation and failures occur naturally.

Environment read/write. The `environment` process can read environment and sensor values and write environment values (cf. Fig. 2 c). For modularization, each environment variable that captures physical dynamics, e.g., rain for an irrigation system, should be modelled in a separate `environment` process. Most importantly, the number of `environment` processes is independent of the number of embedded systems in an IoT system. Since the environment of an IoT system exists as long as the IoT system, the indefinite loop is also in the `environment` process. Often, environments are random, e.g., rain, sun, and wind. Hence, a

change in environment values randomly occurs with dynamics that are computed by the "Changing environment" script task (possibly with ODEs) and written to the corresponding environment variable through the "Set environment" task.

From design time to runtime. Moving the design time DT implemented with our multi-paradigm service pattern to runtime, computation of solutions to ODEs is not required any more, because the real-world embedded systems and the natural environment are available such that the real physical sensors can be connected. Hence, only the `control` process remains with changed endpoints for the "Read" task and the "Control Mode 1" to "Control Mode n" tasks. Instead of getting sensor values from or sending actuator values to the `DT container`, the "Read" task and the n control model tasks in the `control` process of a runtime DT gets sensor values from and actuate the phyiscal device via the embedded system's API respectively. By executing both the design time and runtime DT on equivalent hosts that are connected through an equivalent network, the simulation of the design DT is realistic and the reliability verified on design time DT is expected to hold at runtime. At the same time, the implementation effort to reuse the model from design time is greatly reduced, as the `control` process can be used almost without any changes except changed endpoints to implement and deploy runtime DT. Moreover, any analysis including reliability verification done using the design time DT can be equally done on the runtime DT without changing the analysis. Hence, our multi-paradigm DT service pattern enables seamless integration of design time and runtime for seamless reliability verification.

Example. To illustrate, we consider a simple IoT system of a water kettle and implement Twin-CPEE's DT service pattern at design time yielding a design time DT (cf. Fig. 3). The IoT system consists of a water kettle that contains water. The water temperature inside the kettle shall be between 80° Celsius (C) and 100° C. The `DT container` contains one sensor, two actuator, and four environment variables with their respective values. The sensor variable *temperature* is the kettle's water temperature. The two actuator variables *energy* and *heat* show that the water kettle has 4000 Watt and is turned on. The boolean *heat* actuactor variable models the two control modes of the water kettle. Consequently, the `control` process has two tasks: A "On" task that writes *true* to the *heat* variable and a "Off" task that writes $false$ to the *heat* variable (highlighted in green in Fig. 3). As the water kettle does not control its energy, *energy* is not written and not highlighted in green in Fig. 3. The environment variables *ambient_temperature*, *capacity*, and *mass* show that the room, in which the kettle is located, has a temperature of 25°C and that the water has a capacity of 4186 Joule/(kg*Celsius) and a mass of 1 kg. All three environment variables are required for computing the solution to Newton's law of heating ODE [14]:

$$temperature_{t_{now}} = min(temperature_{t_{last}} + \frac{energy * (t_{now} - t_{last})}{mass * capacity}, 100) \quad (1)$$

Hence, the current temperature $temperature_{t_{now}}$ changes to the minimum of 100°C and the last temperature plus a temperature increase, which is proportional to the energy and passed time during heating, because water temperature

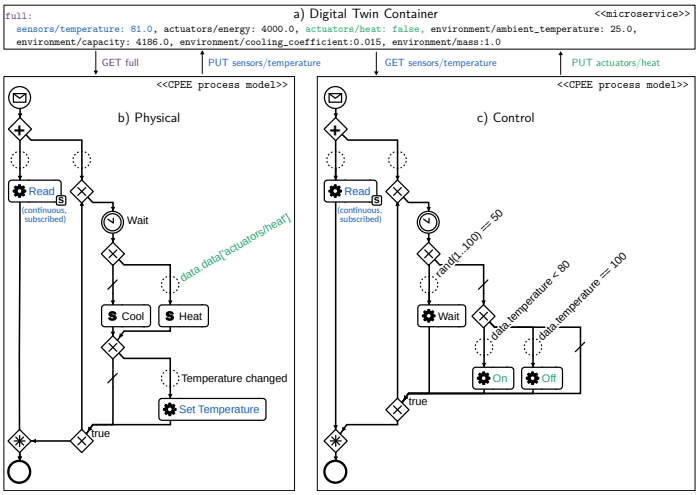

Fig. 3: Twin-CPEE implementation for an example IoT system

does not increase above 100°C. The environment variable $cooling_coefficient$ is required to compute the solution to the ODE for cooling. Hence, the `DT container`'s variables (cf. Fig. 3 a) are sufficient to compute solutions to both ODEs that specify water temperature dynamics during heating and cooling. If the water kettle is turned on, the *heat* variable is *true* (cf. $actuators/heat$ branch in green in Fig. 3b) and the "Heat" script task in the `physical` process computes the new water temperature with Eq. 1 in every execution. Likewise, if the water kettle is turned off, the "Cool" script task computes the new water temperature with the solution formula to the cooling ODE. In both cases, the water temperature changes and is subsequently written to the `DT container` through the "Set Temperature" task in the `physical` process. Lastly, the control can randomly fail with probability of 1% in each control cycle (cf. Fig. 3c). If the control fails, it is delayed through the "Wait" task to model the time of failure and repair.

Overall, the DT service pattern implementation of the water kettle enables reliability verification of how well the water temperature is kept inside the desired range. Given an implemented DT service pattern, we show how reliability verification through SMC is applied to derive statistical assurances on the sensor variable for four different IoT systems in Sect. 4.

4 Evaluation

We demonstrate the Twin-CPEE architecture and implementation with four IoT systems and verify their reliability through SMC.

4.1 Reliability Verification Through Statistical Model Checking

In this section, we show how SMC together with the multi-paradigm DT service pattern (cf. Sect. 3.2) results in reliability verification of DTs. To reduce the complexity of reliability verification, we focus on bounded reliability that verifies over a bounded time period in which the IoT system operates [5]. The IoT system is *reliable* iff it can be proven that it violates a given boundary condition throughout the time period with probability $p < \theta$. As IoT systems aim to maintain a certain function, e.g., temperature or air quality in a building, the function is formalized through the boundary condition, which expresses the desired function as a logical formula over the sensor variables. For example, "maintain the kettle's water temperature above 76°C" specifies the function of the water kettle IoT system (cf. Fig. 3). Assuming a short time period of 20s, the boundary condition is:

$$c_h = temperature \geqslant 76\, \mathcal{U}_{\leqslant 20}\, true \tag{2}$$

The temporal *until* operator is bounded to 20 seconds and refers to an implicit time $t \in \mathbb{R}_{\geqslant 0}$ with positive, real-valued domain. Hence, we take *metric temporal logic*, which is a continuous time variant of *linear time logic*, to formalize boundary conditions. To verify reliability, the boundary condition must be evaluated on a *run* π of the IoT system. For example, a run of the water kettle IoT system is executing the IoT system for 20s such that c_h is either satisfied by π or not. Runs π can be recorded from a model with formal execution semantics at design time, i.e., from the design time DT in Sect. 3.2, or from the IoT system with sensor recordings at runtime, i.e., from the runtime DT. On the one hand, we can apply *probabilistic model checking* [5] that verifies the boundary condition through exhaustive state-space exploration at design time. On the other hand, we can apply SMC to statistically analyse execution runs of the model or real IoT system both at design time or runtime to statistically verify reliability. Since we aim for seamless reliability verification, we opt for SMC to estimate the expected value of the following probability measure on boundary conditions:

$$P(X = 1), \text{where } X(\pi) = 1, \text{if } \pi \text{ violates the boundary condition, else } 0 \tag{3}$$

Because the random variable maps an execution run to either 0 or 1, each execution run is a Bernoulli-trial and the probability of boundary conditions is binomially-distributed [5].

Depending on the input, SMC techniques are either *fixed-k* or *sequential*. Fixed-k SMC techniques take k execution runs and a significance level $\alpha \in [0, 1]$ as input to empirically estimate the expected value $\hat{p}$ of the probability that the boundary condition holds. Hence, $\hat{p} = \frac{\sum_\pi X(\pi)}{k}$ is the empirical estimation that counts how often a run satisfies the boundary condition in proportion to the total number of runs. Additionally, a confidence interval (CI) I with width $|I|$ around $\hat{p}$ is computed. A SMC technique is *sound* iff the real probability p lies within the estimated CI I of width $|I| \leqslant 2\epsilon$, where $\epsilon \in [0, 1]$ is the error, with

probability above $1 - \alpha$ [5]. To illustrate, a typical value for ϵ is 2% and for α is 5%, while k highly depends on the real probability of the IoT system to be reliable. Sequential SMC techniques only take the significance level α and desired error ϵ as input and autonomously compute the necessary number of execution runs k. Lastly, a SMC technique is *inefficient* iff the CI width $|I|$ is either shown to be "unnecessarily" large [5] given k (fixed-k) or the required number of k is "unnecessarily" large (sequential).

Recently, it was shown that many SMC techniques rely on statistical measures that are either unsound, i.e., cannot guarantee statistical assurances for I, or inefficient, i.e., required a larger than necessary k [5,24]. We apply the recent results on soundness and efficiency to quantify statistical assurances for reliability as follows. For the fixed-k setting, we apply the recommended *Clopper-Pearson* interval [7] that is shown to be sound and efficient [24,5] for the probability measure in Eq. 3. For the sequential setting, we apply a similar sequential approach as [5] that precomputes the required k by using the Clopper-Pearson interval. However, instead of precomputing the k by assuming the worst-case probability[4] $p = 0.5$, we assume lower probabilities such as $p = 0.1$, because reliability is typically at least 90% likely. From the proof for soundness of the Clopper-Pearson interval in [24], we conclude that the required k monotonically decreases with lower p. Therefore, the required k we assume is significantly lower than for the worst-case probability.

All in all, applying the two sound and efficient SMC techniques enable reliability verification of IoT systems with statistical assurances on violating boundary conditions. By implementing our multi-paradigm DT service pattern in our service-oriented DT architecture (cf. Fig. 1), reliability verification through SMC is seamless, as the `control` process and, more importantly, its logging does not change (cf. Sect. 3.2). Because the logging service (cf. Sect. 3.1) that logs events while the design time or the runtime DT are executed is the same with the same output, there is no difference between runs from the design time and runtime DT. Moreover, both the design time and runtime DT are implemented with the same architecture, the same pattern, on the same network with equivalent infrastructure. Thus, not only are the two DTs conceptually and technically significantly more similar than with other approaches, but also the two sound and efficient SMC techniques can be seamlessly applied on runs from simulation and execution. Altogether, Twin-CPEE enables seamless reliability verification.

4.2 Reliability Verification Results

In this section, we implement three design time DTs for three IoT systems. Additionally, we consider a runtime DT of a fourth IoT system in [2]. We verify reliability of all four IoT systems with the fixed-k and sequential SMC techniques (cf. Sect. 4.1) as implemented in `https://github.com/janikbenzin/twin-cpee`.

[4] In [24], it is proven that the width of I is largest iff the real probability p equal 0.5. Hence, applying a sequential approach with $p = 0.5$ results in the required k that is always large enough to yield sound statistical assurances.

The four IoT systems were selected to cover all features of our DT service pattern, while balancing complexity, clarity, and comparability.

IoT Systems and Boundary Conditions. The first IoT system is the water kettle (cf. Fig. 3) with c_h. The second IoT system is a water tank with constant water inflow and a pump that can be turned on to prevent overflow and turned off to prevent drying up. The water tank IoT system is based on a common hybrid systems example from [15]. As both overflow and drying up must be prevented, the boundary condition is $c_w = 2 \leqslant level \leqslant 38\ \mathcal{U}_{\leqslant 20}\ true$, where *level* is the water level sensor that measures the level inside the tank in litres. The water tank DT does not have a stochastic fail and repair, but instead contains a noisy sensor, i.e., the level sensor is normally-distributed with the actual *level* as mean and 1l standard deviation. The third IoT system is an agricultural irrigation system that prevents crop on a field from withering. Hence, the boundary condition is $c_i = 0.18 \leqslant moisture\ \mathcal{U}_{\leqslant 24}\ true$, where *moisture* is the soil moisture sensor that measures in the unit $\%/m^3$. Instead of a fail and repair or a noisy sensor, the irrigation DT has five environment processes that model plant transpiration, humidity, temperature, rain, and the hour of a day. Instead of executing over a whole day, we map 1s of execution time in the DT to 1h of the day through the corresponding environment process, i.e., the boundary condition is bound by a day. The fourth IoT system is an automated cotton candy machine [2]. To produce cotton candy and to prevent it from heating unevenly, the boundary condition on the head temperature (°C) inside the cotton candy machine is $c_c = 40 \leqslant head \leqslant 100\ \mathcal{U}_{\leqslant p}\ true$ with p the actual production time in seconds.

Setup and CPEE Execution Properties. Because all DTs are executed on `www.cpee.org` and not simulated, the execution time must be at least the time bound for verifying reliability. Hence, each design time DT is executed for the respective time bound: 20s for the water kettle and water tank DT, 24s for the irrigation DT, and between 15s and 1221s for the cotton candy DT. For the water kettle, we vary the initial water temperature starting with either 81°C, 90°C, or 99°C. For the water tank, we vary the initial water level starting with either 11l, 20l, or 29l. Because the irrigation DT includes random rain throughout the day, we do not additionally vary the initial moisture. For each initial parameter, we execute each design time DT $k = 1000$ times (overall 7000 executions and runs π). Due to the manual effort, the runtime DT is executed $k = 175$ times with initial parameters as measured before execution.

Results. We estimate the probability of unreliability p and the CI through SMC (cf. Sect. 4.1). For fixed-k SMC, we take the available runs per DT and initial parameter respectively to empirically estimate $\hat{p}$ with the surrounding CI. In Tab. 2, the estimated probabilities and CIs are reported. The water kettle DT is not reliable to maintain a water temperature above 76° (cf. c_h) with $\hat{p} = 3\%$. As reliability is concerned with a boundary condition that should not be violated, reliability is derived from the worst-case probability with respect to different initial parameters. Hence, the $\hat{p} = 1.30\%$ and $\hat{p} = 2.30\%$ for the lower and higher initial parameter of 81°C and 99° respectively are not considered. With at least 95% probability, the true probability p lies in the CI [3.00% –

$1.11\%, 3.00\% + 1.11\%] = [1.89\%, 4.11\%]$. The water tank DT is not reliable to maintain a level between 2l and 38l with a probability of 4.20%. The true probability lies in the CI $[2.90\%, 5.50\%]$ with at least 95% probability. Next, the irrigation DT shows the best reliability with a probability of 0.40% for violating the boundary condition and a small CI of $[0.00\%, 0.86\%]$. Finally, the cotton candy DT exhibits the worst reliability with a probability of 8.57% for violating the boundary condition and a large CI of $[4.14\%, 13\%]$.

Table 2: Empirically estimated $\hat{p}$, significance level $\alpha = 5\%$.

Setting	k/ϵ	DT	Condition	$init_{low}$	$init_{mid}$	$init_{high}$
Fixed-k	1000	water kettle	c_h	1.30% ± 0.76%	**3.00% ± 1.11%**	2.30% ± 0.98%
	1000	water tank	c_w	**4.20% ± 1.30%**	0.30% ± 0.41%	0.10% ± 0.28%
	1000	irrigation	c_i	-	**0.40% ± 0.46%**	-
	175	cotton candy	c_c	-	**8.57% ± 4.43%**	-
Sequential ϵ	2.5%	water kettle	c_h	2.19% ± 1.27%	**5.06% ± 1.85%**	3.87% ± 1.64%
	2.5%	water tank	c_w	**7.08% ± 2.15%**	0.51% ± 0.68%	0.17% ± 0.47%
	2.5%	irrigation	c_i	-	**0.67% ± 0.77%**	-
	2.0%	water kettle	c_h	1.42% ± 0.83%	**3.28% ± 1.21%**	2.51% ± 1.08%
	2.0%	water tank	c_w	**4.60% ± 1.41%**	0.33% ± 0.44%	0.11% ± 0.30%
	2.0%	irrigation	c_i	-	**0.44% ± 0.50%**	-

For the sequential SMC, we set two different error values $\epsilon = 2.5\%$ and $\epsilon = 2.0\%$ such that it holds $|I| \leqslant 2\epsilon$ for the CI width $|I|$ (cf. Sect. 4.1). On the assumption of a 10% maximal probability of violating the boundary condition, we compute $k = 593$ and $k = 914$ respectively with the Clopper-Pearson interval. As the cotton candy DT has only 175 runs, we do not estimate probabilities using sequential SMC in this case. In Tab. 2, the estimated probabilities for unreliability using sequential SMC are reported. The water kettle DT, the water tank DT, and the irrigation DT are not reliable with probability $\hat{p} = 5.06\%$, $\hat{p} = 7.08\%$, and $\hat{p} = 0.67\%$ for $\epsilon = 2.5\%$. Consequently, the respective CIs are never larger than 5%. For $\epsilon = 2.0\%$, the respective probabilities decrease to $\hat{p} = 3.28\%$, $\hat{p} = 4.60\%$, and $\hat{p} = 0.44\%$. By assuming a reliability threshold of $\theta = 5\%$ (cf. Sect. 4.1), we conclude that the water kettle and irrigation DT are reliable, while the water tank and cotton candy DTs are not reliable. Since design time DT is executed on the same host (`www.cpee.org`) with the same network as a runtime DT would, the reliability results are expected to also hold for the runtime DT. In addition, implementing the corresponding runtime DT is easy, as only a few minor changes must be applied (cf. Sect. 3.2). Lastly, the design time DT meets the six requirements in Tab. 1 by taking a multi-paradigm approach to combine BPMN with SHS. Next, we present the recording of IoT-enriched logs through the logging service (cf. Fig. 1).

Recording IoT-enriched Logs: As the DT architecture in Fig. 1 contains a logging service, both design and runtime DTs implemented with our service pattern enable recording of IoT-enriched logs in the DataStream XES extension [23]. IoT-enriched logs are event logs that not only contain information on what (script) tasks of the DT are executed at what time, but also what variables are

read or written. An event recorded from executing the water kettle DT (cf. Fig. 3) could, for example, show that script task "Heat" computed a new temperature value of 85.79°C by computing the solution of the Newton's law of heating ODE (cf. Sect. 3.2). By taking the whole IoT-enriched log, log analysis techniques from the process mining community can be applied [12].

5 Conclusion, Limitations, and Future Work

Twin-CPEE as a multi-paradigm DT approach seamlessly integrates a formal model (SHS) to simulate DT behaviour at design time and DT implementation to dynamically represent (BPMN) and control IoT systems at runtime (service orchestration and execution). In particular, Twin-CPEE enables seamless reliability verification through SMC that uses the same logging service at design time and runtime. Twin-CPEE has the following limitations. First, the mode-based representation of control can become hard to understand for a large number of modes. Second, the design time DT of Twin-CPEE requires knowledge on the physical dynamics of the IoT system to determine the ODE and is limited to the ODEs fidelity. Third, Twin-CPEE does not consider *partial* differential equations (PDEs) for which no analytical solution can be found. To improve Twin-CPEE, we will investigate modularization of control modes into subprocesses, discovery of ODEs from sensor time series, and include numerical solutions to PDEs.

References

1. Appel, S., Kleber, P., Frischbier, S., Freudenreich, T., Buchmann, A.: Modeling and execution of event stream processing in business processes. Inf. Syst. **46**, 140–156 (2014)
2. Arteaga Garcia, N., Mangler, J.: Cotton Candy XES YAML (Power Consumption, Environment, Temperatures, Metrology) (2025), `https://zenodo.org/records/17226615`
3. Ballarini, P., Barbot, B., Duflot, M., Haddad, S., Pekergin, N.: HASL: A new approach for performance evaluation and model checking from concepts to experimentation. Performance Evaluation **90**, 53–77 (2015)
4. Barišić, A., , et al.: Multi-paradigm modeling for cyber–physical systems: A systematic mapping review. J. Syst. Soft. **183**, 111081 (2022)
5. Budde, C.E., Hartmanns, A., Meggendorfer, T., Weininger, M., Wienhöft, P.: Sound Statistical Model Checking for Probabilities and Expected Rewards. In: TACAS. pp. 167–190. Springer (2025)
6. Cheng, Y., Zhao, S., Cheng, B., Chen, X., Chen, J.: Modeling and Deploying IoT-Aware Business Process Applications in Sensor Networks. Sensors **19**(1) (2019)
7. Clopper, C.J., Pearson, E.S.: The use of confidence or fiducial limits illustrated in the case of the binomial. Biometrika **26**(4), 404–413 (1934)
8. Corradini, F., Pettinari, S., Re, B., Rossi, L., Tiezzi, F.: A BPMN-driven framework for Multi-Robot System development. Robot. Auton. Syst. **160**, 104322 (2023)
9. Corradini, F., Pettinari, S., Re, B., Rossi, L., Tiezzi, F.: Executable Digital Process Twins: Towards the Enhancement of Process-Driven Systems. Big Data and Cognitive Computing **7**(3), 139 (2023)

10. Dar, K., Taherkordi, A., Baraki, H., Eliassen, F., Geihs, K.: A resource oriented integration architecture for the Internet of Things: A business process perspective. Pervasive and Mobile Computing **20**, 145–159 (2015)
11. David, A., Larsen, K.G., Legay, A., Mikučionis, M., Poulsen, D.B.: Uppaal SMC tutorial. IJSTTT **17**(4), 397–415 (2015)
12. De Luzi, F., Leotta, F., Marrella, A., Mecella, M.: On the Interplay Between Business Process Management and Internet-of-Things. BISE **67**(2), 265–288 (2025)
13. Fedeli, A., Fornari, F., Polini, A., Re, B., Torres, V., Valderas, P.: FloBP: a model-driven approach for developing and executing IoT-enhanced business processes. SoSyM **23**(5), 1217–1246 (2024)
14. Gockenbach, M., Schmidtke, K.: Newton's law of heating and the heat equation. Involve, a Journal of Mathematics **2**(4), 419–437 (2009)
15. Hahn, E.M., Hartmanns, A., Hermanns, H., Katoen, J.P.: A compositional modelling and analysis framework for stochastic hybrid systems. Formal Methods in System Design **43**(2), 191–232 (2013)
16. He, S., He, P., Chen, Z., Yang, T., Su, Y., Lyu, M.R.: A Survey on Automated Log Analysis for Reliability Engineering. ACM CSUR **54**(6), 130:1–130:37 (2021)
17. Jensen, K., Kristensen, L.M., Wells, L.: Coloured petri nets and cpn tools for modelling and validation of concurrent systems. STTT **9**(3), 213–254 (2007)
18. Kwiatkowska, M., Norman, G., Parker, D.: PRISM 4.0: Verification of Probabilistic Real-Time Systems. In: CAV. pp. 585–591. Springer (2011)
19. Lavaei, A., Soudjani, S., Abate, A., Zamani, M.: Automated verification and synthesis of stochastic hybrid systems: A survey. Automatica **146**, 110617 (2022)
20. Lehner, D., et al.: Model-driven engineering for digital twins: a systematic mapping study. SoSyM **24**(5), 1339–1377 (2025)
21. Mandal, S., Hewelt, M., Weske, M.: A Framework for Integrating Real-World Events and Business Processes in an IoT Environment. In: OTM. pp. 194–212. Springer (2017)
22. Mangler, J., Rinderle-Ma, S.: Cloud Process Execution Engine: Architecture and Interfaces. Tech. rep. (2022), `http://arxiv.org/abs/2208.12214`
23. Mangler, J., et al.: Datastream XES extension: Embedding iot sensor data into extensible event stream logs. Fut. Int. **15**(3), 109 (2023)
24. Meggendorfer, T., Weininger, M., Wienhöft, P.: What Are the Odds? Improving the foundations of Statistical Model Checking. Tech. Rep. arXiv:2404.05424 (2025)
25. Mottola, L., et al.: makeSense: Simplifying the Integration of Wireless Sensor Networks into Business Processes. IEEE TSE **45**(6), 576–596 (2019)
26. Pilch, C., Remke, A.: Statistical Model Checking for Hybrid Petri Nets with Multiple General Transitions. In: IEEE/IFIP DSN. pp. 475–486 (2017)
27. Seiger, R., Franceschetti, M., Abbad-Andaloussi, A.: A Process to Non-invasively Augment Legacy IoT Systems Using Business Processes and Microservices. In: ACM IoT. pp. 1–9 (2025)
28. Valderas, P.: Supporting the Implementation of Digital Twins for IoT-Enhanced BPs. In: RCIS. pp. 222–238. Springer (2023)
29. Valderas, P., Torres, V., Serral, E.: Modelling and executing IoT-enhanced business processes through BPMN and microservices. J. Syst. Softw. **184**, 111139 (2022)
30. Voigt, D., Podszun, L., Mangler, J., Rinderle-Ma, S.: HiProX: Highly Efficient Process Execution on IoT and Edge Devices. In: BPM: Responsible BPM Forum, Process Technology Forum, Educators Forum. pp. 184–199. Springer (2026)
31. Wehlitz, R., Rößner, I., Franczyk, B.: Integrating Smart Devices as Business Process Resources – Concept and Software Prototype. In: ICSOC. pp. 252–257. Springer (2018)